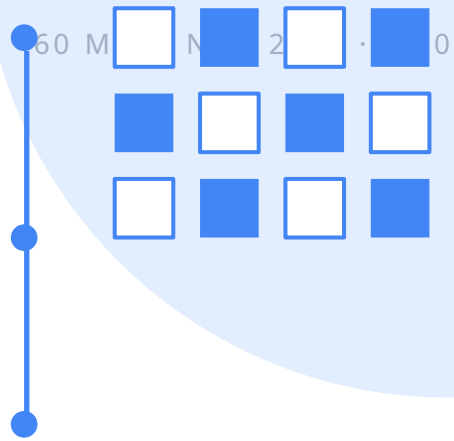


# Using an HPC Cluster: A Working Guide for Computational Physicists

From login node to finished job: the batch workflow you will use every week of your degree.

Lecturer — Janmajay Kumar
Audience — new MSc & PhD students in computational physics
Examples — Slurm on bwUniCluster 3.0; everything generalizes to any Slurm cluster

# Nine topics, one workflow

Concepts first, commands second, failures and habits last — the order in which you will actually meet them.

01 **The mental model** — nodes, scheduler, shared filesystem

02 **Your first batch job** — job script anatomy and lifecycle

03 **Asking for resources** — the five directives that matter

04 **Filesystems** — home, workspaces, and node-local scratch

05 **Software environments** — modules, venvs, and the architecture trap

06 **The command toolkit** — submit, monitor, inspect, cancel

07 **When a job fails** — reading states, exit codes, and logs

08 **Professional habits** — what separates casual users from professionals

09 **bwUniCluster 3.0** — site specifics, partitions, and quirks

# A cluster is many computers, one filesystem, one scheduler

It is not one big machine — and that single fact explains almost every rule you will meet today.

### Login nodes

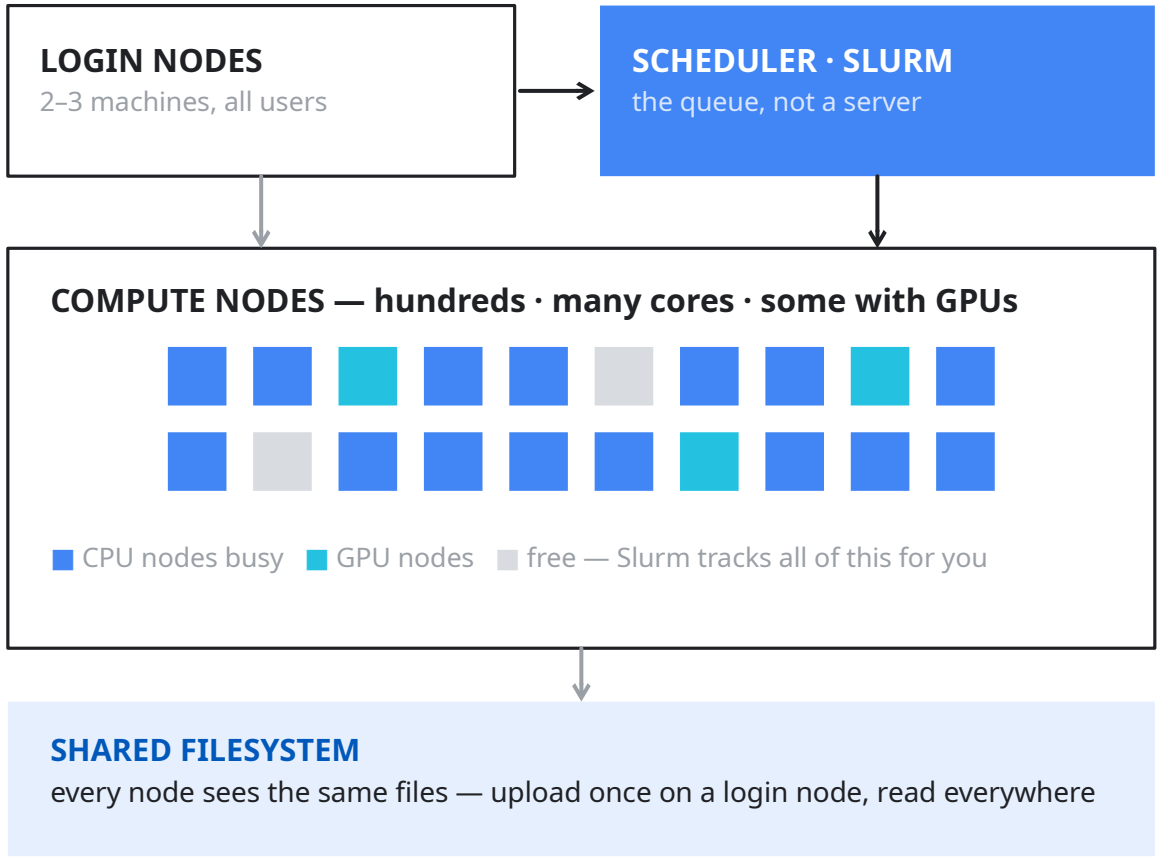
What you reach with ssh. Only two or three exist, and every user shares them. They are for editing, submitting, and housekeeping — with strict per-user limits on CPU and memory. **Never run computation here.** It is the most common beginner mistake, and administrators notice.

### Compute nodes

Where work actually happens. Hundreds of them, each with many cores, large memory, and sometimes GPUs. You cannot log in directly — you request one from the scheduler, which grants it for a bounded period.

### The shared filesystem

Every node sees the same files. Upload data on a login node and read it from a compute node — no copying, no special transfer step.



# Submitting is filing a request, not running a program

You state what you need — one GPU, sixteen cores, four hours — and the scheduler puts you in line. No human is involved.

**1 You submit**
sbatch myjob.sbatch returns a job ID immediately. Nothing has run yet.

→

**2 You wait in line**
State PD. The reason in parentheses says why: Priority, Resources, QOSMaxJobs.

→

**3 Resources free up**
A node matching your request — cores, memory, GPU — becomes available.

→

**4 Your job runs**
State R on a compute node — possibly at 3 a.m., while you sleep.

### The environment reset

· Your job starts in a **fresh shell**, in your home directory
· **No modules** loaded, **no venv** active, no aliases
· Nothing from your interactive session carries over

So: everything the job needs must be stated **inside the job script**.

> "
>
> Most beginner failures are variants of: *it worked when I typed it, but the job didn't have it.*
>
> — the working guide

bwUniCluster and most European academic clusters run **Slurm**. Every command beginning with **s** — sbatch, squeue, sacct, scancel, sinfo — belongs to it. Learn it once, use it at almost any site.

# A job script = shell script + #SBATCH directives

The directives are comments to the shell — but Slurm reads every one of them when you submit.

```
#!/bin/bash
#SBATCH --job-name=nbody-run
#SBATCH --partition=gpu_h100
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=16
#SBATCH --time=04:00:00
#SBATCH --output=logs/run_%j.out

cd "$SLURM_SUBMIT_DIR"
module load devel/python/3.12.3-gnu-14.2
source .venv/bin/activate

./nbody --particles 100000 --steps 5000
```

`sbatch myjob.sbatch` → `Submitted batch job 38421057` — you are back at your prompt immediately. Nothing has run yet.

**1** **cd "$SLURM_SUBMIT_DIR" comes first**

The job starts in your home directory, not where you submitted. Without this line, every relative path in the script is wrong — the classic reason a venv "cannot be found".

**2** **%j expands to the job ID**

Concurrent jobs never overwrite each other's logs — each run gets its own output file.

**3** **logs/ must exist beforehand**

Slurm opens the output file before your script runs. A missing directory kills the job instantly, with no useful message. Run mkdir -p logs first.

**4** **The script is the whole environment**

Modules and venv activation belong inside the script, every time. Nothing is inherited from the shell you submitted from.

# Jobs outlive your session — that is the point of batch

Close the laptop, lose the VPN, fly somewhere — an accepted job keeps going, and its output lands in the log file.

| State | What it means |
|---|---|
| `PD` | Pending — waiting in line; the reason in parentheses says why: Priority, Resources, QOSMaxJobs |
| `R` | Running — the TIME column counts up |
| `CG` | Completing — cleaning up, usually seconds |
| `gone` | Vanished from squeue = finished, successfully or not — check with sacct |

**PD (Resources)** — your request is valid but the hardware is busy.
**PD (Priority)** — other jobs are ahead of you.

Neither is an error. A job that queues for hours is normal cluster life.

**Survives logout**

Batch jobs submitted with `sbatch` run on a compute node independently of your session. Output streams to the log file while you sleep.

**Dies with your shell**

`salloc` and `srun --pty` give interactive allocations tied to your terminal. Use them for debugging — never for real runs.

The job ID is your handle for everything that follows:

```
squeue -j 38421057 · sacct -j 38421057 · scancel 38421057
```

# Five directives cover almost every resource request

The scheduler enforces exactly what you ask for — no more, no less. Describe the machine you need; Slurm finds it.

| Directive | What it sets | What goes wrong |
|---|---|---|
| `--partition` | Which named pool of nodes | Wrong partition is the most common instant rejection. List with sinfo -s, inspect with scontrol show partition. |
| `--time` | Wall-clock limit, HH:MM:SS | At expiry the job is killed mid-computation — no grace period, no checkpoint unless you wrote one. Ask for ~2× your estimate. |
| `--cpus-per-task` | CPU cores for the job | Extra cores help only if the code can use them (OpenMP threads, MPI ranks). Your integrator gets no faster from cores it never spawns. |
| `--mem` | Memory, whole job | On per-GPU partitions memory is allocated automatically — adding --mem there triggers “Requested node configuration is not available”. |
| `--gres=gpu:1` | One GPU | A GPU partition alone is not enough — without --gres your job cannot touch the GPUs on its own node. |

One mental model: you are **describing a machine**. Slurm either finds it, queues you until it exists, or rejects the request outright.

# Over-asking costs queue time; under-asking costs the job

The scheduler must find a window where everything you asked for is free at the same time.

### ASK FOR TOO MUCH

All of it must be free simultaneously — the bigger the request, the longer the scheduler searches for a matching window.

Cost: a few extra hours in the queue. An annoyance.

### ASK FOR TOO LITTLE

Exceed the request and the job is killed immediately — mid-computation, no grace period, no warning.

**Cost: the whole job, lost at hour eleven of twelve. A day gone.**

## Be generous on --time, realistic on everything else.

### Backfill rewards short jobs

A short job can jump the queue if it fits into a gap before a large reserved job starts. A one-hour job often starts immediately where a twelve-hour job waits — a real reason to **split a long pipeline into stages**.

```
sbatch --test-only --partition=gpu_h100 --gres=gpu:1 --time=04:00:00 --wrap="hostname"
```

Queues nothing — reports an estimated start time, or names the exact resource it cannot satisfy. Check before you commit.

# Three filesystems, three very different contracts

Using the wrong one is a slow-burning mistake — everything works until suddenly nothing does.

### `$HOME`

small · backed up · permanent

Source code, scripts, notes, small results. Limited in total size **and** in number of files — the second limit catches people out: a single Python virtual environment holds tens of thousands of small files and can exhaust the file-count quota on its own.

### `workspace / scratch`

large · fast · parallel · not backed up

Datasets, checkpoints, intermediate output, virtual environments. On bwHPC, workspaces are created **with an expiry date** — extend before it lapses, or the data is deleted. Called workspace on bwHPC, scratch on many other sites.

### `/tmp on the node`

node-local · fast · erased at job end

Scratch space on the compute node itself. Useful for files a single job writes and rereads heavily — nothing survives the end of the job.

**Rule of thumb** — small and irreplaceable → **$HOME** · large and regenerable → **workspace** · upset to lose a workspace → it needs a **real backup** elsewhere.

# Workspaces expire; quota errors crash jobs mid-run

Two slow-burning mistakes: letting a workspace lapse, and hitting a quota limit halfway through a week-long run.

### Workspace commands (bwHPC)

```
ws_allocate myproject 60   # 60 days
ws_find myproject          # prints the path
ws_list                    # what you have, expiry dates
ws_extend myproject 60     # extend before it lapses
```

`cd $(ws_find myproject)` — the `$( )` runs the command and substitutes its output. That is why you see this idiom in every bwHPC example.

### Check your quota before large operations

```
lfs quota -uh $USER /pfs/data6   # Lustre
du -sh ~/some-directory          # what a dir costs
```

Running out of quota mid-job produces write errors that surface as **confusing application crashes**, often far from the real cause.

Check before extracting a large archive — and before starting a long run that writes checkpoints every few thousand steps.

Workspaces are **not backed up** — and that is meant literally. Keep anything irreplaceable in $HOME or an external archive; treat workspace data as regenerable or temporary.

# No sudo, no apt — software comes from modules

You cannot install system packages. Environment modules are pre-built software you load into your shell on demand.

```
module avail                            # everything (long)
module avail 2>&1 | grep -i python  # filtered
module spider pytorch                   # incl. hidden versions
module load devel/python/3.12.3-gnu-14.2
module list                             # what is loaded now
module purge                            # unload everything
```

**Two useful habits.** `module spider` finds versions that `module avail` hides.

And record the exact module names **in your job script** — not in your memory, not in a wiki page you will lose.

### What module load actually does

It edits environment variables — `PATH`, `LD_LIBRARY_PATH`, `PYTHONPATH`. It installs nothing and deletes nothing.

`module purge` likewise only unloads; it cannot destroy files.

**Loaded modules do not persist into your jobs.**

Every job script must load what it needs — every time, explicitly.

# Build your own venv — and respect the CPU architecture

Clusters provide Python but not the scientific stack, so you build an environment of your own — once, correctly.

```
module load devel/python/3.12.3-gnu-14.2
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

1. **Load the same module in every job.** The venv's python3 is a symlink into that interpreter — purge the module and the symlink points nowhere.
2. **Build the venv in a workspace, not $HOME** — the file-count quota again.
3. **Install on a login node.** Compute nodes often have no outbound internet — pip inside a job fails or hangs.

### THE ARCHITECTURE TRAP

Compiled wheels — NumPy, SciPy, PyTorch — carry CPU instructions chosen at install time. A wheel installed on an AVX-512 node can die with **Segmentation fault (core dumped)** on an older node: no traceback, no error message, just a dead process.

### Two signatures to recognise

· Works on one node, segfaults on another → check `hostname` on both

· `import numpy` alone segfaults → rebuild the venv

```
python3 -X faulthandler -c "import numpy"
```

Bites hardest where one login reaches two generations of hardware — see bwUniCluster's UC2 trap, later in this lecture.

# A small command set covers almost everything

Group them by what you are trying to do — squeue watches the live queue, sacct reports on jobs that have left it.

### Submitting

```
sbatch job.sbatch # prints a job ID
sbatch --array=0-19%5 job.sbatch
sbatch --dependency=afterok:12345 b.sbatch
sbatch --export=ALL job.sbatch # pass env in
```

### Monitoring

```
squeue -u $USER # your jobs, their state
squeue -u $USER --start # start-time estimate
squeue -j 12345 -o "%T %R" # state + reason
```

### Inspecting

```
sacct -j 12345 --format=JobID,State,Elapsed,MaxRSS,ExitCode
seff 12345 # efficiency summary
scontrol show job 12345 # everything Slurm knows
```

### Cancelling

```
scancel 12345 # one job
scancel 12345_3 # one array task
scancel -u $USER # everything you have
```

**sacct** is the one to reach for after a job vanishes from squeue — **MaxRSS** is the peak memory actually used; compare it with your request and adjust next time.

# Arrays for sweeps, salloc for debugging, rsync for data

Three everyday operations, three idioms worth copying straight into your own notes.

### Parameter sweeps → job arrays

`sbatch --array=0-19%5 job.sbatch` runs 20 tasks, at most 5 at once. Inside, `$SLURM_ARRAY_TASK_ID` tells each task which slice it owns — one task per temperature in an Ising sweep, per initial condition in an N-body ensemble. Chain pipeline stages with `--dependency=afterok:12345`.

### Interactive work → debugging only

`salloc --partition=dev_gpu --gres=gpu:1 --time=00:30:00`, then `srun --pty bash`, gives you a shell on a compute node. It dies with your terminal — use it to debug, never for real runs. Always check `hostname` once you land: which node, and which hardware generation?

### Moving data → run rsync from the machine that has the file

`rsync -avP file.tar.gz user@cluster:/path/to/destination/` — whichever side you type on, the *other* side gets the `user@host:` prefix; `-P` resumes a broken transfer. Verify anything large with `sha256sum` on both sides: a truncated HDF5 opens without complaint and gives you wrong numbers silently.

# Gone from squeue means finished — not succeeded

A job vanishing from the queue tells you it ended. Whether it worked is a separate question — always check.

### The 30-second post-mortem

```
sacct -j 12345 --format=JobID,State,Elapsed,MaxRSS,ExitCode
# COMPLETED with 0:0 is success.
# Anything else → the logs:
tail -50 logs/myjob_12345.out
cat  logs/myjob_12345.err
```

`seff 12345` gives an efficiency summary where available — CPU and memory utilisation at a glance.

A job at 8% CPU efficiency on sixteen cores was **waiting on something** — usually input, sometimes the filesystem. That is a performance finding sitting in your logs for free.

| State / signal | Usual cause |
|---|---|
| `FAILED, short Elapsed` | Script error: bad path, missing module, venv not found |
| `TIMEOUT` | --time too short; resubmit longer, with checkpointing |
| `OUT_OF_MEMORY` | Exceeded --mem; check MaxRSS, request more |
| `CANCELLED` | You, an administrator, or a failed dependency |
| `NODE_FAIL` | Hardware problem — not your fault; just resubmit |
| `Exit 139` | Segmentation fault — compiled library, often architecture mismatch |
| `Exit 137` | Killed by signal 9 — usually the out-of-memory killer |

# The five failures you will actually hit

Learn to recognise them on sight — each becomes a two-minute fix instead of a lost afternoon.

1. **Dies in under a second**

   The environment: missing `cd "$SLURM_SUBMIT_DIR"`, an unloaded module, or a venv path that does not resolve. The .err file names it.

2. **“Requested node configuration is not available”**

   Your request fits no node in that partition — usually `--mem` on a per-GPU partition, or more cores than a node has. `scontrol show partition <name>` shows the real limits.

3. **torch.cuda.is_available() is False**

   You forgot `--gres=gpu:1`, or you are on a login node, or you installed a CPU-only wheel. `nvidia-smi` inside the job distinguishes the three.

4. **The output file never appears**

   The directory in `--output` did not exist — Slurm opens the file before your script runs, so the job dies writing nothing anywhere. `mkdir -p logs` before submitting.

5. **Works interactively, fails in the job**

   The canonical batch bug. Your shell had modules, a venv, and a working directory; the job inherits none of them. Everything goes in the script.

**A habit worth forming — start every job script with diagnostics**

```
echo "job $SLURM_JOB_ID on $(hostname) $(date)"
nvidia-smi --query-gpu=name --format=csv,noheader || true
```

→ the top of every log tells you which node, which GPU, and whether the environment came up at all.

# Habits that separate casual users from professionals

None of these are about being clever — they are about not losing days.

**01 Probe first**

Before a twelve-hour job, run the same code for twenty minutes on a development queue with a tiny input. A probe that fails in two minutes has saved you a day.

**02 Measure, then extrapolate**

Time a small run, compute seconds per unit of work, multiply. That gives you a defensible --time instead of a guess — and a cost estimate before you commit.

**03 Read MaxRSS and CPU efficiency**

After every significant job. 8% CPU on sixteen cores means you were waiting on something — usually the input pipeline, sometimes the filesystem.

**04 Make jobs resumable**

Write checkpoints; let each stage skip itself if its output already exists. A pipeline you can resubmit after a crash is worth the hour it takes to write.

**05 Smallest first in a sweep**

Order runs so partial results stay usable. Four finished small runs are a figure in your thesis; one half-finished large run is nothing.

**06 Record provenance**

Git commit, hostname, library versions, date — written into the output file itself. When a reviewer asks which solver version produced a dataset, you will have the answer.

**07 Script everything, including fixes**

A hand edit on the cluster exists nowhere else. Put it in a script or in version control, or it will be lost the next time you rebuild.

**08 Be a good citizen**

No computation on login nodes; no idle allocations; arrays with a throttle instead of thousands of one-minute jobs. The person you inconvenience is usually in your own department.

And a ninth, from the data-transfer section: verify anything large with **sha256sum on both sides** — silent corruption is the worst failure mode.

# bwUniCluster 3.0: access, partitions, limits

Site specifics for the cluster you will actually use — everything before this slide generalizes to other Slurm sites.

### Logging in

```
ssh -4 tu_<your-id>@uc3.scc.kit.edu
```

· Access is restricted to **BelWü network addresses** — connect the university VPN first if you are off campus
· Authentication: **TOTP code**, then your service password
· Tübingen usernames carry the `tu_` prefix; KIT's use `ka_`

| Partition | Hardware | Max time |
|---|---|---|
| `cpu` | AMD, 96 cores/node | long |
| `cpu_il` | Intel Ice Lake, 64 cores | long |
| `highmem` | large-memory nodes | long |
| `gpu_h100` | 4× H100 per node | 72 h |
| `gpu_h100_il` | 4× H100, Ice Lake host | 72 h |
| `gpu_a100_il` | 4× A100 | 48 h |
| `*_short · dev_*` | same hardware | 30 min |

GPU queues allocate memory per GPU automatically — gpu_h100: **24 cores and ~193 GB per GPU**; gpu_a100_il: 16 cores and ~127 GB. **Do not pass --mem** on them. And avoid gpu_mi300 unless you have a ROCm build — a standard CUDA wheel will not run there.

# Site quirks worth learning before they cost you hours

Four bwUniCluster specifics you will not find in generic Slurm documentation.

### sinfo is restricted

Access/permission denied from `sinfo` is expected — use `sinfo_t_idle` instead.

There is no account or project to specify either: omit `--account` entirely.

### The UC2 trap

One login reaches **two hardware generations**, UC2 and UC3. A venv built on UC3 can segfault on UC2 nodes — the architecture trap in practice.

Check `hostname`: `uc3n…` good, `uc2n…` wrong generation. Pin it: `#SBATCH --clusters=uc3`.

### No PyTorch module

Load Python, build your own venv, then `pip install torch --index-url https://download.pytorch.org/whl/cu124`.

The wheel bundles its own CUDA runtime — **do not** also load devel/cuda; mixing the two causes library confusion. devel/miniforge exists if you prefer conda.

### gpu_mi300 needs ROCm

The AMD MI300A partition exists, but a standard CUDA build will not run on it.

Unless you have a ROCm build of your code, stay on the **H100 / A100 partitions**.

Authoritative source: the bwHPC wiki at **wiki.bwhpc.de** — the site changes faster than any guide, including this one.

# Your first week on the cluster

Five steps, in order. Each one is small; together they are the whole workflow.

**1**

**Log in and look around**

sinfo -s (or sinfo_t_idle), module avail, ws_list. Nothing to break.

**2**

**Set up your environment**

A venv in a workspace; the module names recorded in your job script, not your memory.

**3**

**Submit a 5-minute test job**

Include the diagnostics header at the top of the script; mkdir -p logs first.

**4**

**Read the outcome**

squeue while it runs; sacct and the logs after. Compare MaxRSS with what you requested.

**5**

**Scale the physics**

Probe, measure, extrapolate — then submit the real run, with checkpointing.

> "You are not running a program. You are filing a request that a program be run — in an environment you fully control, on hardware you share with your colleagues.
>
> — the mental model from the first section

**Further reading** — wiki.bwhpc.de (bwHPC wiki, bwUniCluster 3.0 section) · slurm.schedmd.com (Slurm documentation)

Questions now — and good luck with the simulations.